Artificial Intelligence
Innovative Project

GatedKT: An Adaptive Maths Assessment System with a Gated Knowledge-Tracing Model

by

Reg No	Team Member Names
24BCE0229	Sarthak Maheshwari
24BDS0344	Ananya Ambastha

Under the Supervision of

Prof. Annaporna

Assistant Professor Sr. Grade-2

School of Computer Science and Engineering

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

November 2026

Contents

1. Abstract
2. Introduction
3. Novelty in Project
4. Proposed Work
5. Implementation & Screenshots
6. Conclusion
7. Project Repository Link

References

1. Abstract

Online practice usually gives every student the same fixed quiz and reduces the result to one score. That score does not tell a teacher which topic a student is weak in, or at which level of difficulty. This project builds **GatedKT**, a complete adaptive assessment system for school mathematics. At its centre is a neural knowledge-tracing model that predicts, before every question, the probability that the student will answer it correctly.

The model keeps two memories of the student: a short-term GRU over the last 10 answers and a long-term GRU over the whole history. A learned vector gate decides, for each of its 64 dimensions, how much to trust each memory. The gate also uses a graph of related skills (graph attention over a PPMI co-occurrence graph) and the student's behaviour: response time, number of attempts and hints. The prediction drives an adaptive quiz that chooses the next topic and level, offers hints and a second attempt, and produces a per-topic mastery profile and a teacher dashboard.

On the ASSISTments 2009 dataset (3,119 students, 338,210 answers, 123 skills), the full gated model reaches a test AUC of **0.8713** on 313 held-out students. It is statistically better than four standard baselines: DKT (GRU) 0.8583, SAKT-lite 0.7973, BKT 0.7819 and PFA 0.7657. Across 50 retraining seeds the learned gate improved accuracy on **50 of 50** matched runs. The system is a working web application with a Python model service, a Node.js gateway and 289 passing automated tests.

Keywords: knowledge tracing, GRU, graph attention network, gated fusion, adaptive assessment, educational data mining.

2. Introduction

Every student learns at a different pace, but one teacher often looks after forty or more students and cannot follow each of them question by question. A fixed quiz spends questions on topics the student already knows and hides the real gap.

Knowledge tracing (KT) addresses this by modelling what a student currently knows from the sequence of answers they have given, and predicting the probability that the next answer will be correct (Corbett & Anderson, 1994). Bayesian Knowledge Tracing (BKT) models each skill as a hidden binary state. Performance Factors Analysis (PFA) uses counts of past successes and failures in a logistic model (Pavlik et al., 2009). Deep Knowledge Tracing (DKT) replaced these with a recurrent neural network over the whole answer sequence (Piech et al., 2015). Later work added self-attention (SAKT; Pandey & Karypis, 2019) and graph neural networks over related skills (Nakagawa et al., 2019).

Two issues remain in most of this work. First, recent behaviour and long-term history are merged into a single state, so a model cannot easily react to a streak or a slump while still remembering what the student learned earlier. Second, the prediction is rarely used: it is reported as an accuracy number, not used to choose the next question or to tell a teacher where the gaps are.

2.1 Objectives

1. Build a KT model that combines a short-term memory, a long-term memory, a skill-relation graph, question difficulty and behaviour signals, joined by a learned gate.
2. Evaluate it against standard baselines (PFA, BKT, DKT, SAKT) on the same split, with statistical tests and many random seeds.
3. Use the prediction inside an adaptive quiz that chooses topic and difficulty before every question, with hints and a second attempt.
4. Give students a per-topic mastery profile and teachers a class dashboard.
5. Deliver a working, tested web application.

3. Novelty in Project

1. **Learned per-dimension gate between two time scales.** A short-term GRU (re-run on the last 10 answers at every step) and a long-term GRU (whole history) are mixed by a 64-dimensional gate. The gate is conditioned on skill-graph context and behaviour signals. Removing the gate lowers test AUC on all 50 of 50 matched seeds.
2. **Difficulty-aware prediction.** Question difficulty (Easy, Medium, Hard, derived from training data only) is an input, so the model can answer "what is P(correct) on this topic at each level?". This is used for a mastery profile that is certified only where the student was actually tested near that level.
3. **Hints and attempts as model inputs.** The app records hint requests and second attempts, and feeds them back to the model. After a plain wrong answer the predicted chance on the next question is 0.41; after a wrong answer that needed two tries it is 0.58.
4. **Prediction drives decisions.** The prediction is made and logged before the student sees each question. It selects the next topic, and each question carries a visible reason built from the model's numbers.
5. **Measured vs predicted knowledge.** Topics never asked are shown as "predicted, not measured" using the skill graph, and are checked with probe questions.
6. **Rigorous evaluation.** Four baselines on the same split, a 50-seed variance study, paired tests with Holm correction, 1,000-sample bootstrap intervals, pre-registered experiments and an automated audit that traces every reported number to a result file.

4. Proposed Work

4.1 Dataset and preprocessing

We use the public ASSISTments 2009 skill-builder dataset. Students with fewer than 10 interactions are removed. Students are split 80/10/10 into training (2,495), validation (311) and test (313) sets (seed 42), so no student appears in two sets. Within each skill, problems with at least five training responses are ranked by correct-answer rate and cut into thirds, giving difficulty buckets 1 (Easy), 2 (Medium) and 3 (Hard). Both the difficulty buckets and the skill graph are built from training students only.

4.2 System architecture

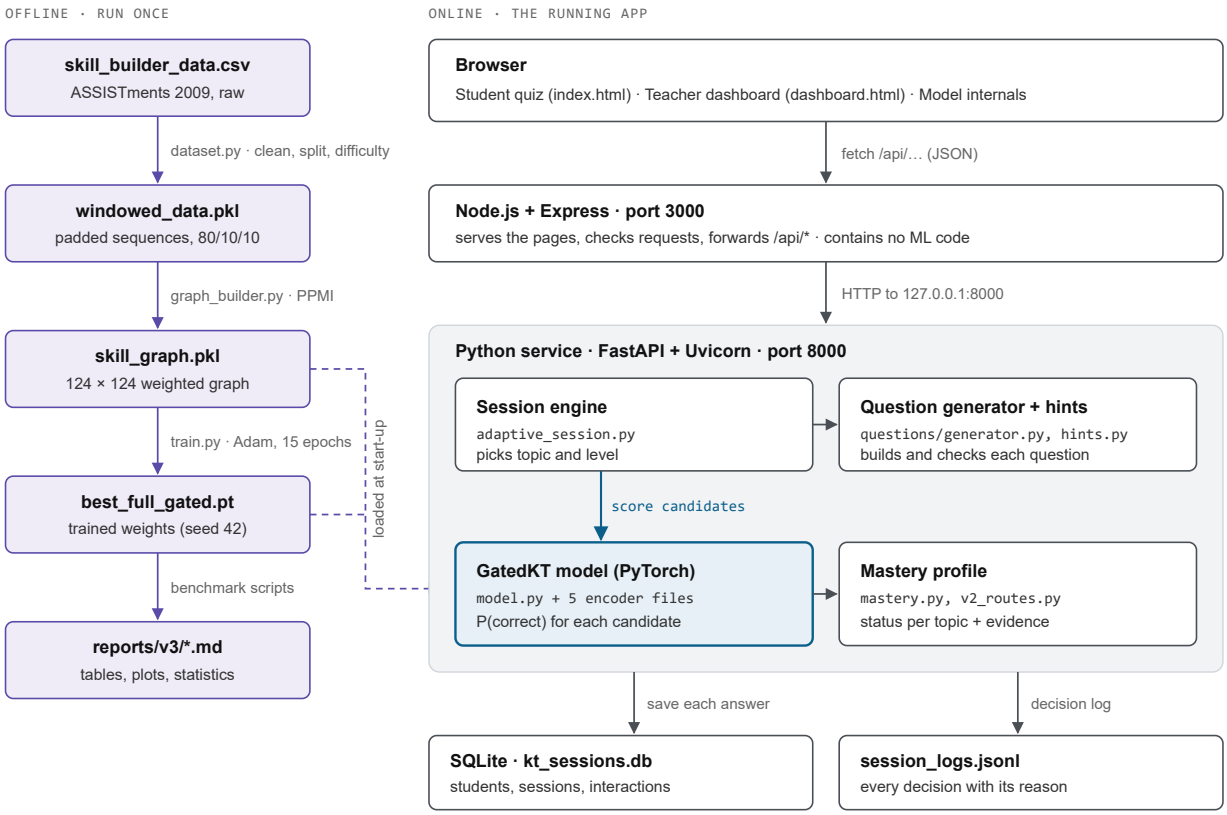


Figure 1. System architecture. Purple: offline pipeline (run once). Right: the running application. The service loads the skill graph and the trained weights at start-up.

The offline pipeline cleans the data, builds the skill graph, trains the model and writes evaluation reports. The online application has three tiers: a browser front end, a Node.js/Express gateway (port 3000) that serves the pages and forwards API calls, and a Python FastAPI service (port 8000) that holds the model, the session engine, the question generator and the mastery module. Answers are stored in SQLite and every decision is logged with its reason.

4.3 Model architecture

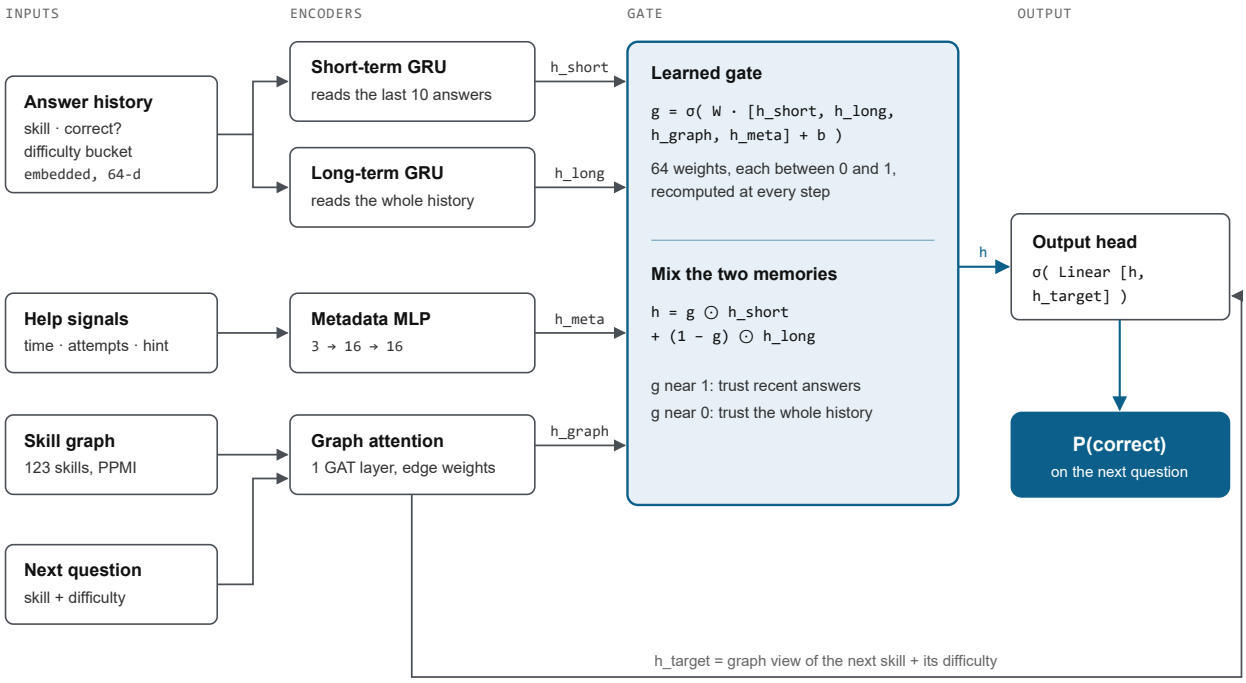


Figure 2. GatedKT model. The two memories are combined only in the learned gate. The skill graph gives the gate context about the next skill and tells the output head which skill is being predicted.

At each step t, the computation is:

$$
\begin{aligned}
x_t &= E_skill[s_t] + E_correct[c_t] + E_diff[d_t] && \text{(64-d input)} \\
h_short &= GRU_short(\, x_(t-9) \ldots x_t \,) && \text{(last 10 answers)} \\
h_long &= GRU_long(\, x_1 \ldots x_t \,) && \text{(whole history)} \\
m_t &= MLP(\, \log(1+rt_t),\, attempts_t - 1,\, hint_t \,) && (3 \to 16 \to 16) \\
h_graph &= GAT(\, E_skill,\, PPMI\ graph \,)[s_(t+1)] \\
g_t &= \sigma(\, W_g\, [h_short \,\|\, h_long \,\|\, h_graph \,\|\, m_t] + b_g \,) && \text{(64 gate values)} \\
h_t &= g_t \odot h_short + (1 - g_t) \odot h_long \\
\hat{y}_(t+1) &= \sigma(\, W_o\, [h_t \,\|\, h_graph + E_diff[d_(t+1)]] + b_o \,)
\end{aligned}
$$

Skill graph. Skills that appear in the same session window are linked. Co-occurrence is converted to positive pointwise mutual information, PPMI(a,b) = max(log(P(a,b) / (P(a)P(b))), 0). Pairs seen together in fewer than 5 windows are dropped, giving 1,880 directed edges among 123 skills. A single-layer graph attention network (Veličković et al., 2018) refines every skill embedding from its neighbours, using the PPMI weight as an edge feature.

Gate. A gate value near 1 trusts recent answers; near 0 it trusts the whole history. Because it is a vector, different dimensions of knowledge can be weighted differently. The served model has

76,497 trainable parameters.

4.4 Training and calibration

The loss is binary cross-entropy over real (non-padded) answers. Training uses Adam (learning rate 10^{-3}), batch size 32, up to 15 epochs, keeping the weights with the best validation AUC, with seeds 42, 123 and 2024. Probabilities are then calibrated by temperature scaling (Guo et al., 2017), $p = \sigma(\text{logit}(p) / T)$ with $T = 1.167$ fitted on validation data. This lowers the expected calibration error from 0.0249 to 0.0117 without changing AUC.

4.5 Decision layer

- **Topic selection:** the model's prediction for each eligible topic is blended with the student's own record on it, score = $(2 \cdot \hat{y} + \text{correct}) / (2 + \text{answered})$, and the weakest topic is asked next (at most 4 questions per topic, no more than 2 in a row).
- **Difficulty:** a staircase per topic (correct moves one level up, wrong one level down). An uncertainty-aware policy uses an ensemble of three checkpoints and falls back to the staircase when they disagree.
- **Hints and retries:** a hint can be requested and a wrong answer can be retried once. As in the dataset, an answer counts as correct only if it was right on the first try without a hint.
- **Mastery profile:** for each topic the model gives P at Easy, Medium and Hard (made monotone). Statuses (Needs support, Developing, Proficient, Mastered) use a 70% threshold plus evidence rules: at least 2 answers, and a level is certified only where the student was tested near it.

5. Implementation & Screenshots

5.1 Technology stack

Layer	Technology	Use
Model	PyTorch 2.14, PyTorch Geometric 2.8	GRUs, embeddings, gate, GATConv layer, training and inference
Data	pandas 3.0, NumPy 2.4	Cleaning, splitting, padding, graph construction
Evaluation	scikit-learn 1.9, SciPy 1.17, Matplotlib 3.11	AUC, paired tests, bootstrap, plots
Service	FastAPI 0.143, Uvicorn, Pydantic 2	Python API on port 8000
Gateway	Node.js, Express 4	Serves the pages, forwards `/api/*` calls
Front end	HTML, CSS, JavaScript, Chart.js, vis-network	Quiz, results, teacher dashboard, skill-graph view
Storage	SQLite, JSONL log	Students, sessions, answers, decision log
Testing	pytest, Node test scripts, Git	289 passing automated tests; version history

Table 1. Technology stack.

5.2 Module structure

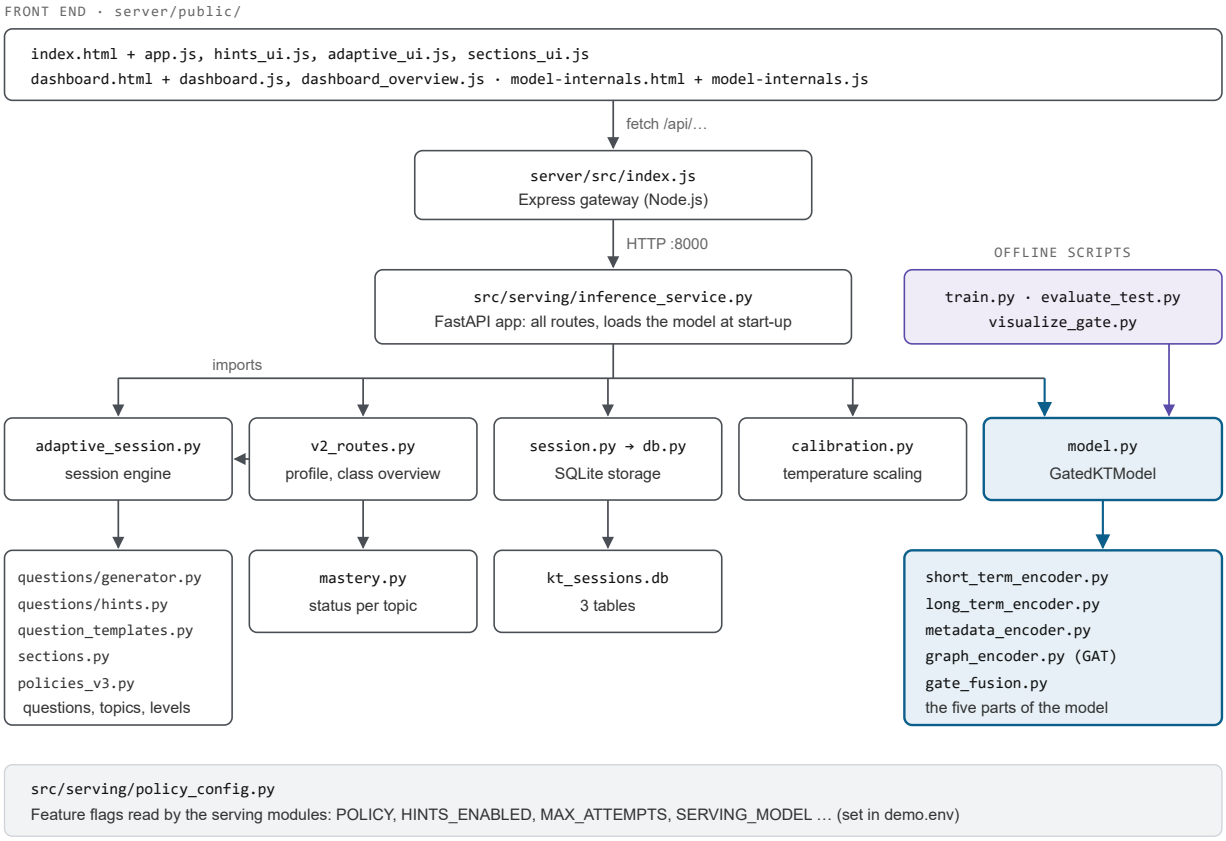


Figure 3. Module connectivity, drawn from the import statements in the code. The training scripts import the same model.py that is served.

5.3 Results

All models are evaluated on the same 313 held-out test students. AUC is the probability that the model gives a correct answer a higher score than a wrong one (0.5 is guessing).

Model	Type	Seeds	Test AUC	Brier	Full gated vs model
GatedKT, full gated (ours)	**proposed**	**3**	**0.8713**	**0.1344**	**—**
GatedKT, graph without gate	**ablation**	**3**	**0.8657**	**0.1370**	**better, +0.0056**
DKT (GRU)	**deep baseline**	**10**	**0.8583**	**0.1408**	**better, +0.0130**
SAKT-lite	**attention baseline**	**10**	**0.7973**	**0.1689**	**better, +0.0740**
BKT	**classical**	**50**	**0.7819**	**0.1715**	**better, +0.0894**
PFA	**classical**	**50**	**0.7657**	**0.1810**	**better, +0.1055**

Table 2. Comparison with four baselines. "Better" means a Holm-corrected paired Wilcoxon test with $p < 0.05$ and a bootstrap 95% confidence interval of the AUC difference that excludes 0.

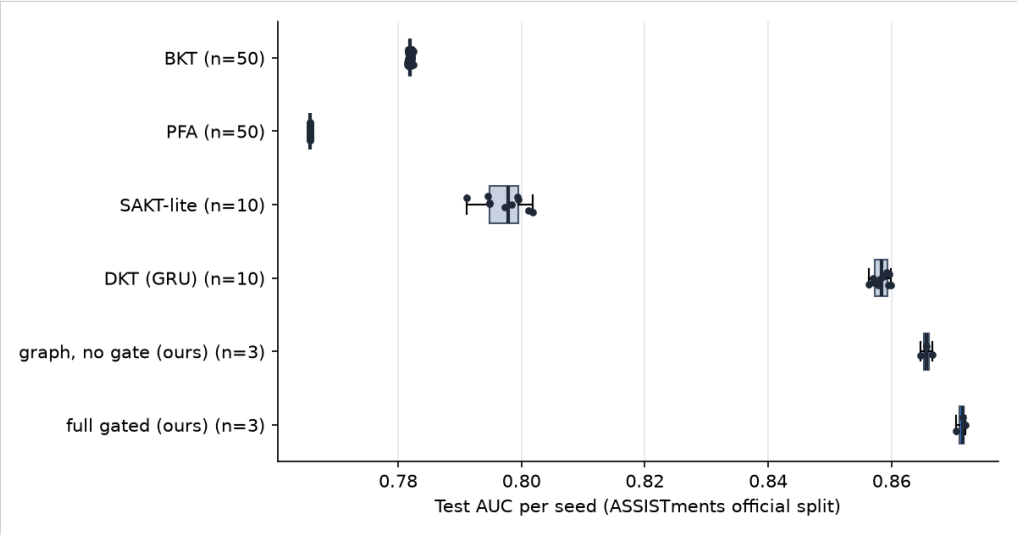


Figure 4. Test AUC of every trained run. Each dot is one seed; the full gated model (bottom) is furthest right.

Variant (50 seeds, same budget)	Mean AUC	Std	Min	Max	Gain of full gated [95% CI]
Full gated	**0.8532**	**0.0015**	**0.8503**	**0.8559**	**—**
DKT-only (no graph, no gate)	**0.8469**	**0.0018**	**0.8430**	**0.8513**	**+0.0062 [+0.0058, +0.0067]**
Graph without gate	**0.8459**	**0.0017**	**0.8421**	**0.8501**	**+0.0073 [+0.0070, +0.0077]**

Table 3. Ablation over 50 seeds (5 epochs with early stopping). The full gated model scored higher on 50 of 50 matched seeds; the gain is 4–5 times the seed-to-seed spread.

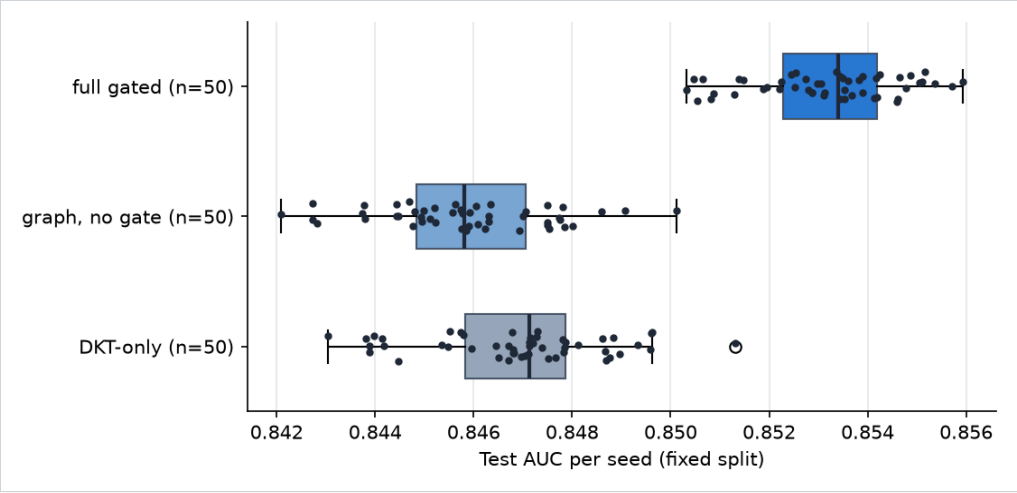


Figure 5. Fifty runs per variant: every full gated run is above every run without the gate.

Check	Result
Difficulty ordering P(Easy) ≥ P(Medium) ≥ P(Hard)	**Held in 100% of 600 validation and 6,000 quiz-topic cases**
Calibration (expected calibration error)	**0.0249 → 0.0117 with T = 1.167; AUC unchanged**
Effect of hints and attempts on the next prediction	**wrong, no help 0.41; wrong after hint 0.45; wrong with 2 attempts 0.58; correct 0.77**

Table 4. Consistency, calibration and behaviour checks.

5.4 Screenshots of the working system

The screenshots below were taken from one live session of the running application (student "report_demo_student", 16 questions).

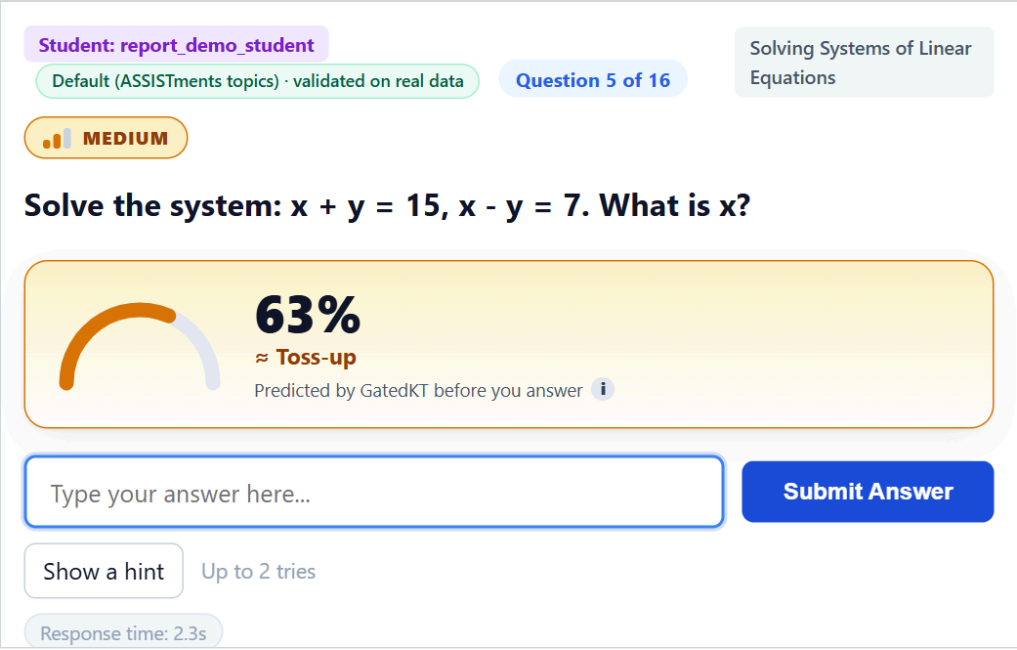


Figure 6. Quiz screen. Before the student answers, GatedKT predicts a 63% chance of success on this Medium question. The hint button and "up to 2 tries" are shown.

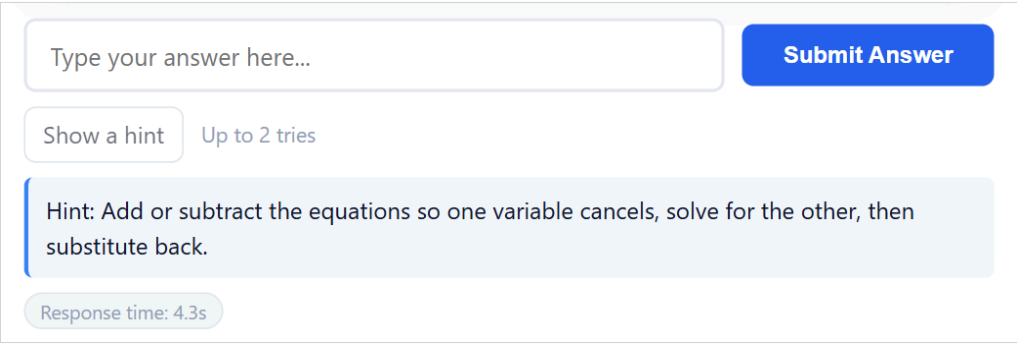


Figure 7. A strategy hint for the topic. Using the hint is recorded and fed to the model.

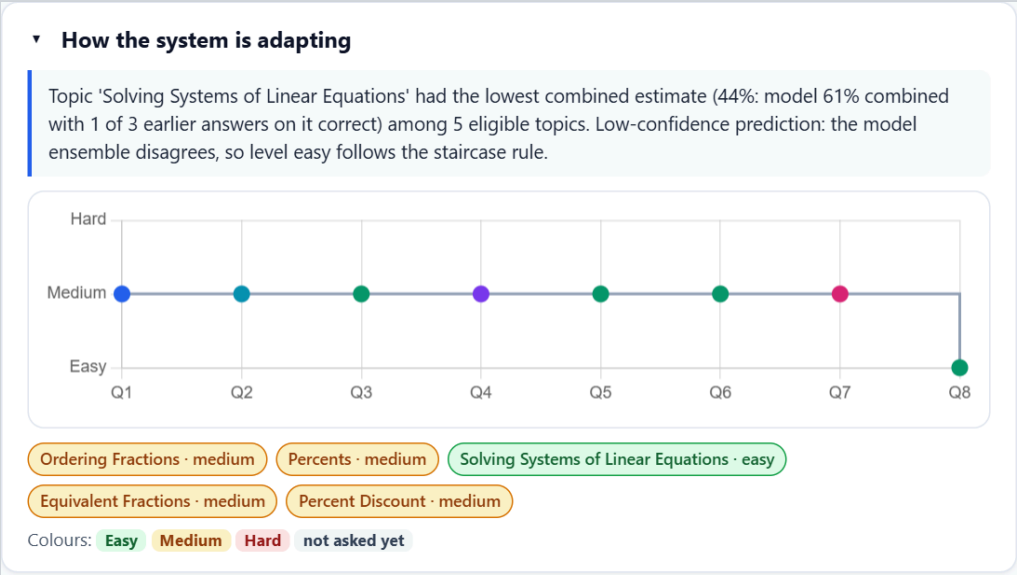


Figure 8. "How the system is adapting": the reason for the chosen topic (built from the model's numbers), the difficulty path across questions and the level of each topic.

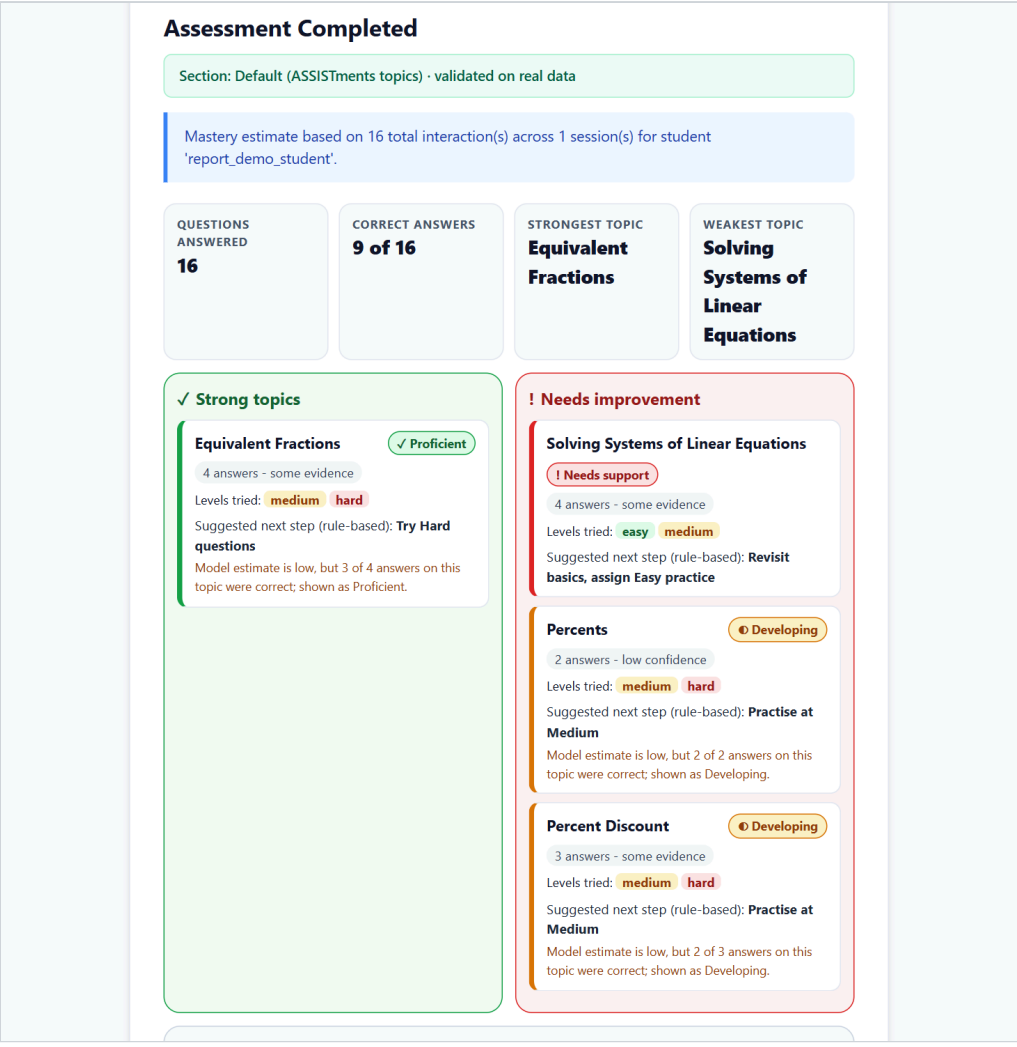


Figure 9. End-of-session report: strong topics and topics needing improvement, each with evidence and a suggested next step.

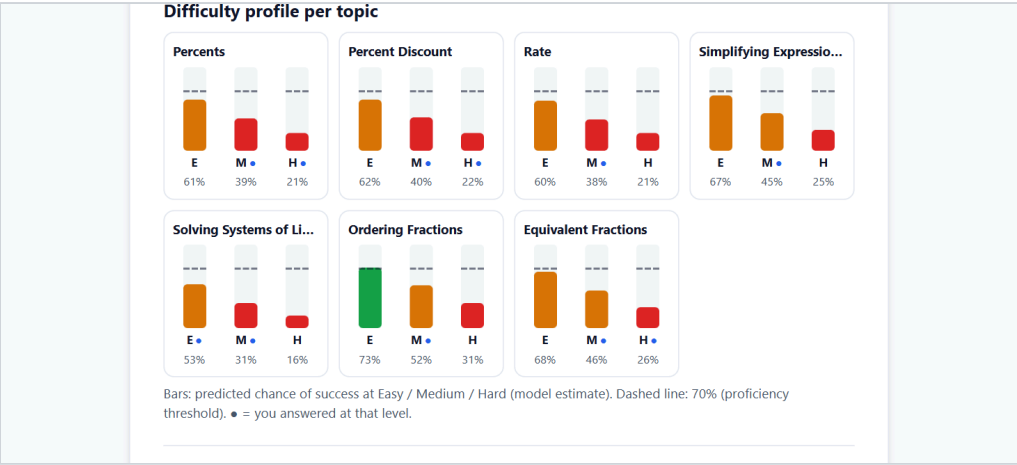


Figure 10. Difficulty profile per topic: predicted chance of success at Easy, Medium and Hard against the 70% proficiency line.

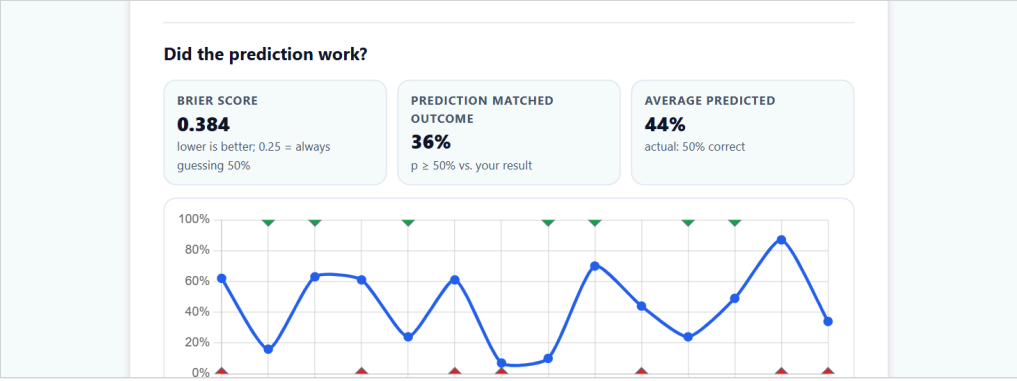


Figure 11. "Did the prediction work?": the prediction made before each answer compared with the actual result for this session.

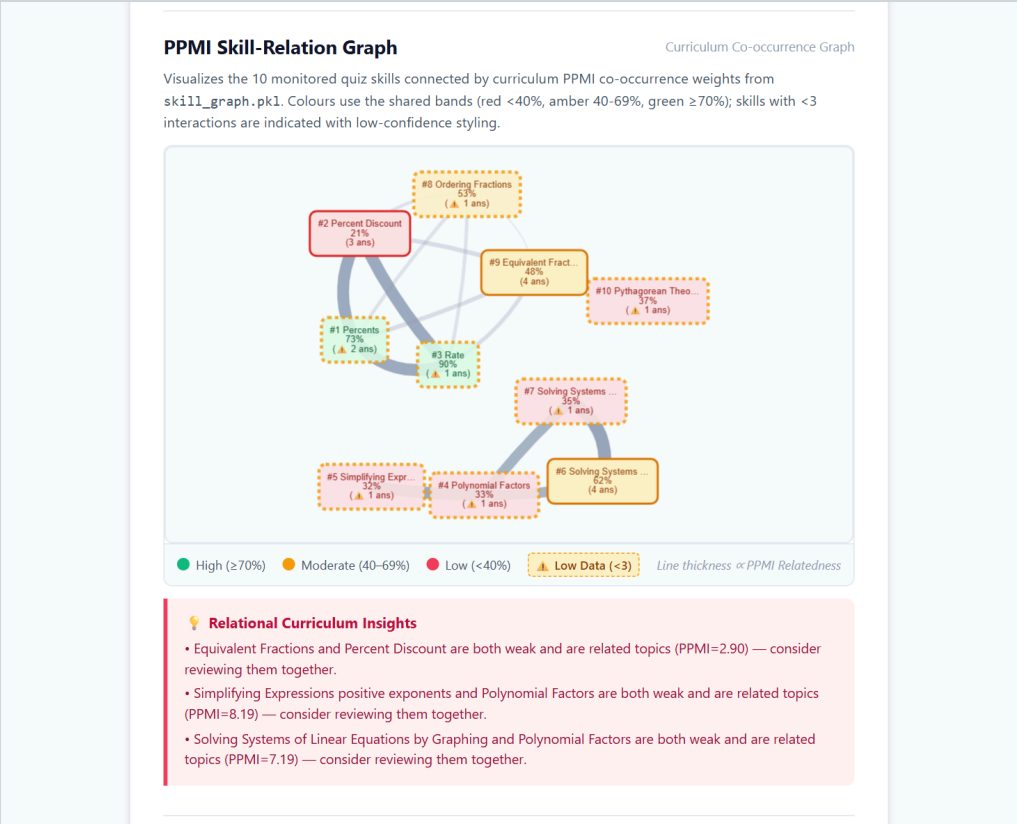


Figure 12. PPMI skill-relation graph coloured by the student's mastery, with related weak topics flagged.



Figure 13. Teacher dashboard: student × skill mastery matrix from live data.

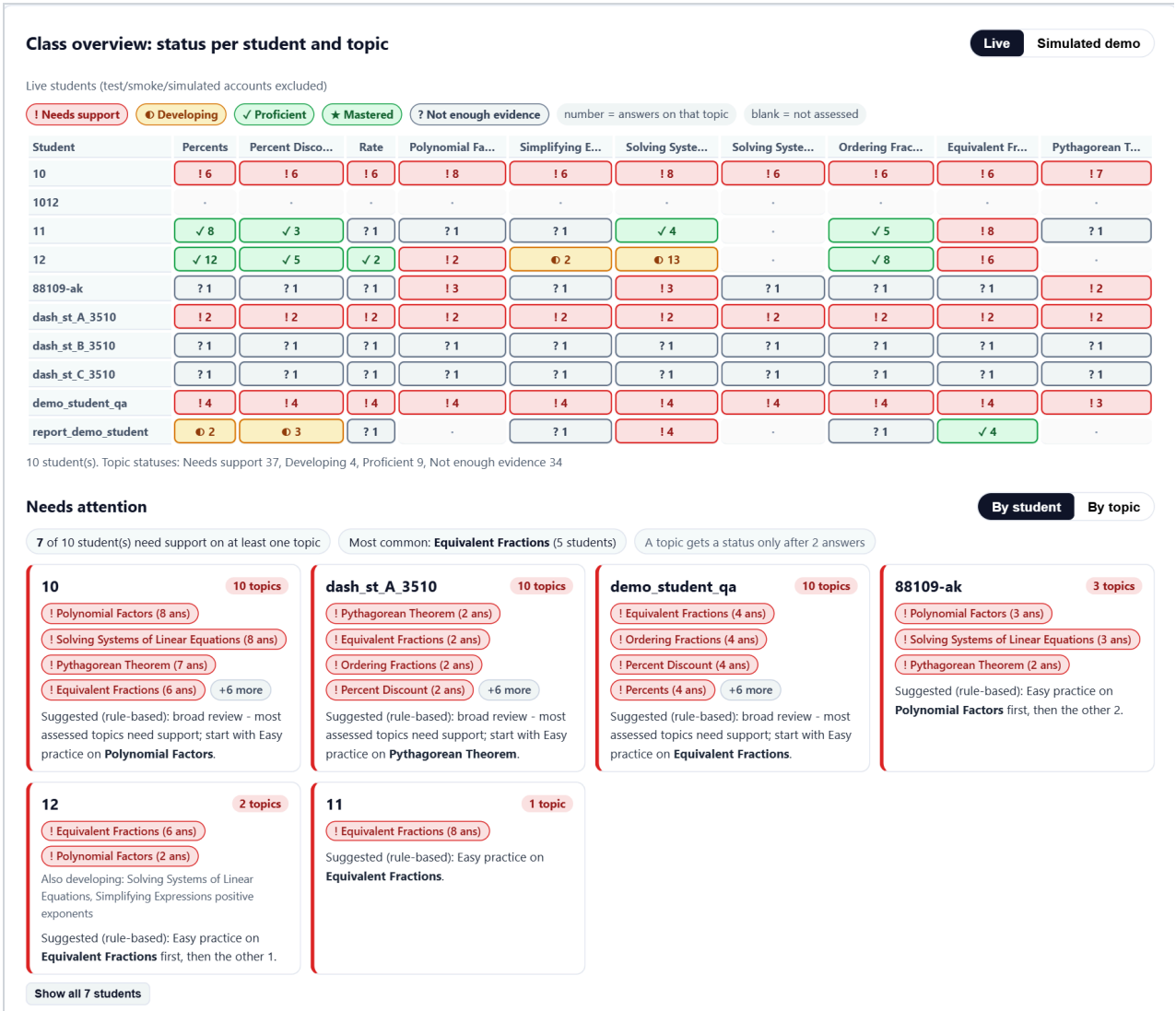


Figure 14. Class overview (status per student and topic) and the needs-attention list with rule-based suggestions.



Figure 15. Model internals: the learned gate at every step of the session (top) and per dimension (bottom), showing how the model shifts between short-term and long-term memory.

6. Conclusion

GatedKT turns knowledge tracing from an offline accuracy number into a working adaptive assessment. The model combines short-term and long-term memories through a learned per-dimension gate, conditioned on a skill-relation graph and on behaviour signals including hints and attempts. On ASSISTments 2009 it reaches a test AUC of 0.8713, is statistically better than DKT, SAKT, BKT and PFA, and the gate's contribution holds on all 50 of 50 matched seeds. The predictions are calibrated, respect the difficulty ordering in every checked case, and are used before every question to choose the topic and level.

The complete system, with a quiz offering hints and retries, a per-topic mastery profile, a teacher dashboard and a model-internals view, runs as a tested three-tier web application. Future work includes evaluating on further datasets such as EdNet, comparing with more attention-based

models such as AKT under the same split, and running a classroom study with real students and teachers.

7. Project Repository Link

GitHub: **https://github.com/ananyaa25-prog/Intelligent-Knowledge-Tracing**

The repository contains the data pipeline, the model and training code, the PPMI graph builder, the baselines, the evaluation scripts and result files, the FastAPI service, the Node.js gateway, the web application and the automated tests.

References

Corbett, A. T., & Anderson, J. R. (1994). Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction, 4*, 253–278. https://doi.org/10.1007/BF01099821

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning* (pp. 1321–1330). PMLR 70.

Nakagawa, H., Iwasawa, Y., & Matsuo, Y. (2019). Graph-based knowledge tracing: Modeling student proficiency using graph neural network. In *Proceedings of the IEEE/WIC/ACM International Conference on Web Intelligence* (pp. 156–163). https://doi.org/10.1145/3350546.3352513

Pandey, S., & Karypis, G. (2019). A self-attentive model for knowledge tracing. arXiv:1907.06837.

Pavlik, P. I., Cen, H., & Koedinger, K. R. (2009). Performance Factors Analysis: A new alternative to knowledge tracing. In *Proceedings of the 14th International Conference on Artificial Intelligence in Education* (pp. 531–538). IOS Press.

Piech, C., Bassen, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L. J., & Sohl-Dickstein, J. (2015). Deep knowledge tracing. In *Advances in Neural Information Processing Systems, 28*, 505–513.

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph attention networks. In *International Conference on Learning Representations*.

Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of EMNLP 2014* (pp. 1724–1734).

Feng, M., Heffernan, N. T., & Koedinger, K. R. (2009). Addressing the assessment challenge with an online system that tutors as it assesses. *User Modeling and User-Adapted Interaction, 19*, 243–266. https://doi.org/10.1007/s11257-009-9063-7